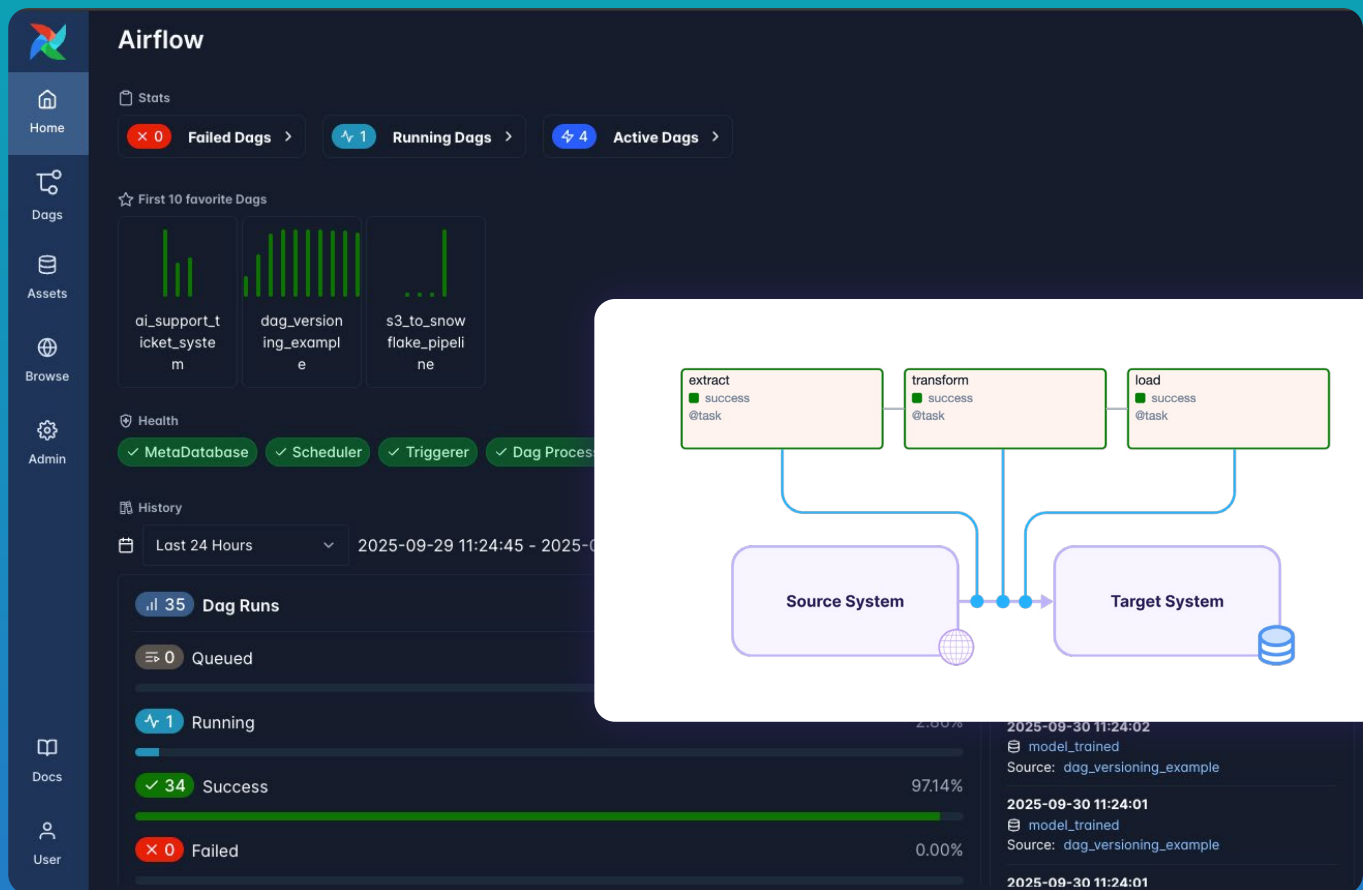




Apache Airflow® 3 Best Practices

ETL and ELT Pipelines

ASTRONOMER

Table of Contents

Definitions	4
What is ETL?	4
What is ELT?	5
ETL and ELT in real-world scenarios	6
Pre-flight checklist for ETL and ELT pipelines	7
Key decisions for ETL and ELT pipelines	8
ETL vs ELT	9
Choose a method for passing data between tasks	10
Airflow's compute vs external compute	13
Task-oriented vs asset-oriented approach to writing pipelines	14
Task-oriented approach: choice of operators and decorators	20
Data Quality Checks	21
Airflow best practices for ETL and ELT	24
Three core DAG writing best practices	24
ETL DAG design best practices	25
Avoid top-level code	25
Automatic retries	26
Use Software Development Lifecycle best practices	27
Testing Airflow code	28
Unit Tests	29
DAG validation tests	30
Scaling	32
Airflow environment best practices	32
Key Airflow features for ETL and ELT	33
Dynamic Task Mapping	33
Generate DAG code programmatically	36
Modularized task groups	38
Data-driven scheduling	41
Airflow Object Storage	44
Other important features for ETL and ELT	45
Key Airflow providers for ETL and ELT	46
Major Cloud Providers	46
Transformation tools	46
Data warehouses and data lakes	47
Airflow for ETL/ELT checklist	48
Why choose Astronomer for ETL and ELT	49
Conclusion	50

Introduction

The field of data engineering has seen rapid transformation in recent years, with an ever-expanding array of tools, diverse use cases, growing stakeholder demands, and the addition of generative AI, all contributing to the evolving skill set data engineers are expected to develop and excel in.

Regardless of the end use case, whether for analytics, complex dashboards, or fine-tuning large language models, your data must first be **extracted (E)** from its source, **transformed (T)** into a target schema, and **loaded (L)** into your central data storage, whether that's a data warehouse or a data lake.

ETL and ELT pipelines form the foundation of any data architecture, making the ability to design, build, and understand these pipelines essential for every data engineer.

Originally developed in 2014 specifically for ETL and ELT pipelines, [Apache Airflow®](#) has since evolved into the open-source standard for full-stack orchestration of any workflow, without losing sight of its roots in moving and transforming data.

According to the [2025 State of Apache Airflow Report](#), over 85% of current Airflow users rely on it to run ETL and ELT pipelines, powering everything from some of the world's largest organizations to fast-growing startups.

This eBook is your comprehensive guide to building modern ETL and ELT pipelines with Apache Airflow, featuring the latest tools and techniques. Whether you're starting fresh or enhancing an existing setup, it covers everything from conceptual overviews and pipeline planning to essential features and integrations.

At Astronomer, we hope this eBook serves as a valuable resource, and we can't wait to see what you'll build with it. <3

helpful hints

If you have never used Airflow before, we recommend you start with our **Get Started** tutorial before reading this eBook.

[Complete the Tutorial](#)

See the companion GitHub repository for example code showcasing the patterns mentioned here.

[Check out the Repo](#)

To try out the concepts explained in this eBook in a cloud-environment, start a **free 14-day Astro trial**.

[Start for Free](#)

Definitions

Before you dive into writing code, it is important to define the patterns used, as well as talk about how they are implemented in real-life scenarios.

What is ETL?

ETL stands for **Extract (E), Transform (T), Load (L)**. It's the original pattern designed to move data from a source system, transform it, and load it into a target system, commonly a data warehouse or data lake. The defining feature of ETL is that the **transformation** happens “in-flight,” meaning data is processed and formatted before being loaded into the target system. This allows for control over which parts of the data are persisted and in what format.

Three common reasons to apply a transformation step before loading your data are:

- **Selective Persistence:** You may only want to store a fraction of the extracted data in your target system. For instance, when pulling data from an API that delivers large datasets, you might only need a handful of fields, like total orders for each customer, for your use case.
- **Data Enhancement and Combination:** You pull in data from different sources and combine them before loading into your target system.
- **Format Compatibility:** Your source data may be in a format that your target system cannot accommodate directly. For example, if you're working with image or video data for machine learning purposes, it could be useful to create embeddings in the transformation step before loading them into a vector database. However, this scenario is becoming less common with the increasing support for unstructured data in many data lakes.

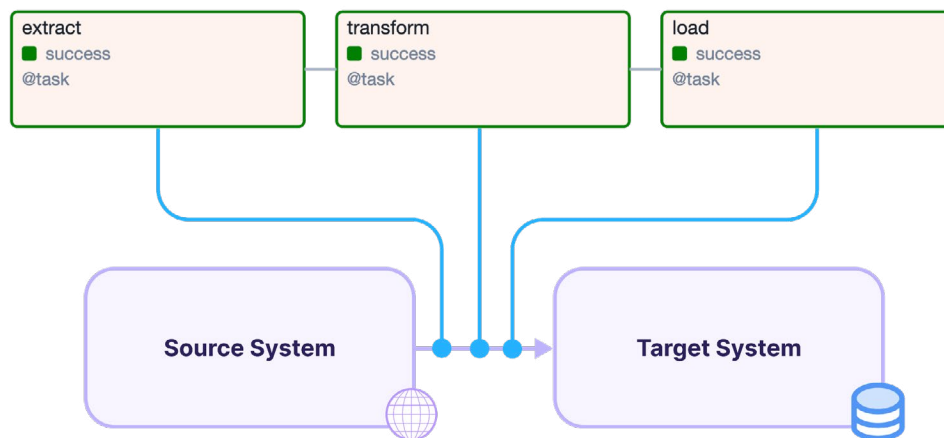


Figure 1. Architecture diagram of a simple ETL pattern (for full code examples see: [ETL pattern DAG using XCom](#) and [ETL pattern DAG using explicit external storage](#)).

What is ELT?

As data lakes and data warehouses have evolved in recent years, the ability to ingest unstructured data directly has improved and storage costs have decreased. These developments have led many organizations to adopt an **ELT (Extract, Load, Transform)** pattern.

In an ELT pattern, the full raw data is loaded directly from the source system into the target system before any transformations take place. Once loaded, transformations are applied within the target system to create a second data asset, often in the form of a relational table.

Organizations commonly choose an ELT pattern in the following situations:

- **Easy Access to Full Raw Datasets:** They want to store and retain all the raw data in the target system for current or potential future use.
- **Use Target System's Compute Power:** Many data warehouses and data lakes offer parallel processing and close-to-data compute capabilities, also known as pushdown compute. Pushdown compute refers to performing transformations within the target system itself, allowing for faster and more efficient processing of complex transformations without needing to process the data externally.

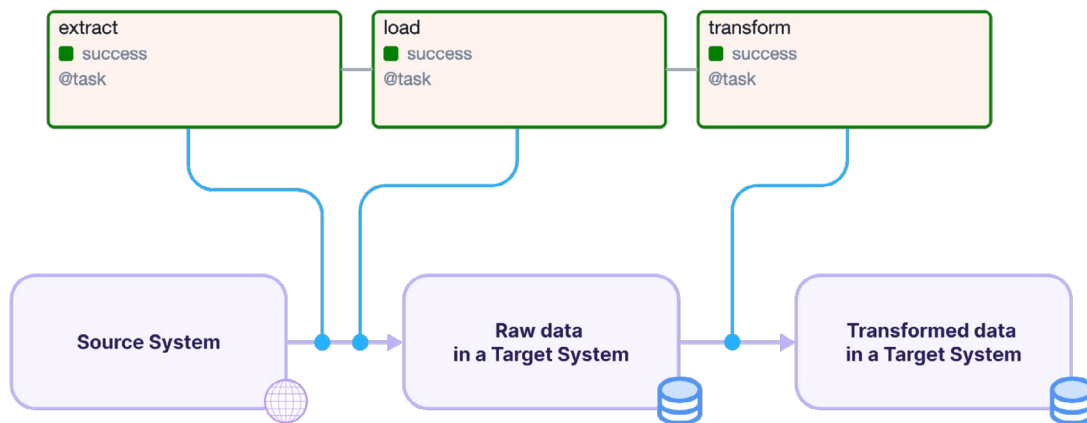


Figure 2. Architecture diagram of a simple ELT pattern (for full code examples see: [ELT pattern DAG using XCom](#) and [ELT pattern DAG using explicit external storage](#)).

ETL and ELT in real-world scenarios

While much has been written about the differences between ETL and ELT, the reality is that many organizations use a combination of both approaches, tailored to their specific data sources, often merging them into an ETLT pattern. In this pattern, extracted data is initially transformed in-flight, loaded, and then transformed again, often in several steps, to create assets for different downstream use cases.

Fortunately, Apache Airflow offers the flexibility to orchestrate these mixed patterns side by side in an observable and interconnected way.

At the end of the day, we all have data that needs to be moved from one place to another and eventually must be presented in a format that's accessible and usable for our stakeholders. Whether an ETL or ELT pattern is used is often secondary to ensuring that the end result is a reliable and trusted asset for downstream use, creating happy stakeholders and a good night's sleep for us data engineers.

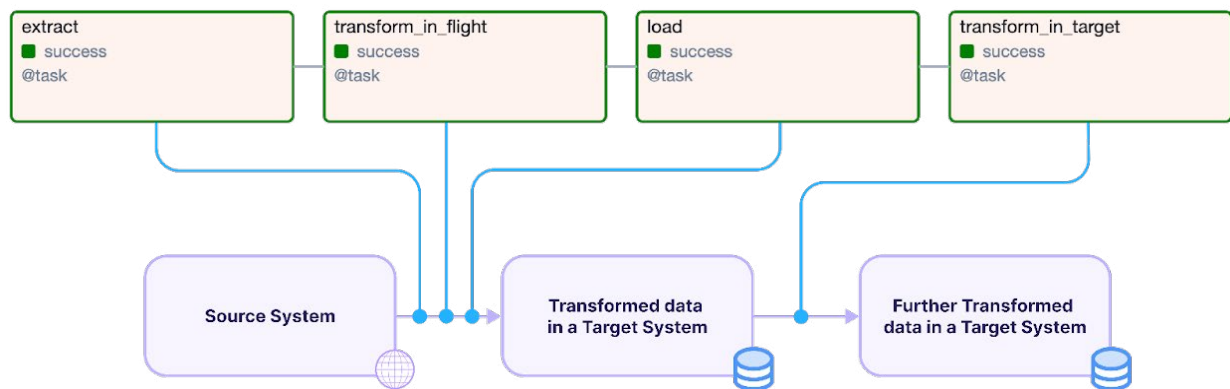


Figure 3. Architecture diagram of a simple ETLT pattern (for a full code example see: [ETLT DAG using XCom](#)).

Pre-flight checklist for ETL and ELT pipelines

Before diving into writing code, ensure you have answers to the following key questions to make solid architectural decisions and avoid rework later.

☐ Stakeholder Requirements

- ☐ Who are the data consumers, and what are their use cases?
- ☐ How do they access the data?
- ☐ What specific data is needed, what are the freshness requirements?

☐ Data Quality & Governance

- ☐ Are there data contracts with upstream/downstream users?
- ☐ What data quality issues should stop the pipeline vs raise an alert?
- ☐ Are there data privacy and data access concerns?

☐ Source System

- ☐ What is the location, format, size, and update frequency of the data?
- ☐ Are data quality issues present?

☐ External storage in between ETL steps

- ☐ Is external storage required? What are the options?
- ☐ Do you need to archive raw data? What are the cleanup/retention policies?

☐ Target System

- ☐ What data will be persisted, and where?
- ☐ Are there existing schema conventions or formatting requirements?

☐ Transformation Needs

- ☐ How will you transform the data to match the target schema?
- ☐ What are the compute and storage requirements for transformations? Do you need distributed processing?

☐ Existing structures

- ☐ What tools are already approved and in use at your organization?
- ☐ What other pipelines do you need to interact with, up- and downstream?

Answering these questions often involves collaboration across multiple teams, each with potentially conflicting priorities. Some questions may remain unresolved, requiring data leaders to strike a balance between delivery timelines, pipeline efficiency, and future needs.

Key Decisions for ETL and ELT Pipelines

Now that you gathered as much information about your organization's desired state as possible, likely with some gaps to fill in, you are ready to make some high-level decisions about the pipeline architecture, namely:

- Using **ETL vs ELT** vs a **combination** of both
- Passing data through Airflow's **XCom** vs using **external storage** that is explicitly accessed
- Transform the data using **Airflow's compute vs external compute**
- Deciding whether to use the task-oriented or asset-oriented approach to writing your pipelines.
- If you are using the task-oriented approach, deciding what kind of **operators and decorators** to use
- Deciding what kind of **data quality checks** to implement

The above decisions influence and are influenced by the tools you choose to use in conjunction with Airflow, typically using existing tooling if possible.

ETL vs ELT

With Airflow it is possible to combine the patterns ETL and ELT freely. For example, you can use ETL for one data source and ELT for the next, or use an ETLT pattern, transforming the data in-flight before loading and transforming it again in the target system before the data is being used by your stakeholders.

The table below summarizes the differences between ETL, ELT, and ETLT, helping you to decide which approach best suits your needs.

	ETL	ELT	ETLT
Source data format	Any format	Formats that can be loaded directly into the target system	Any format
Source data batch size	Limited by Airflow worker memory	No limit if the target system supports distributed loading	Limited by Airflow worker memory
Data persistence in the target system	Parts of the data (or all of the data)	All of the data	Parts of the data (or all of the data)
Data enhancement before loading into the target system	Possible	Not possible	Possible
Compute for transformation	Airflow worker (or external tool)	Target system (data warehouse, data lake)	Two step compute possible: typically first in the Airflow worker, then in the target system
Transformation language	Primarily Python (other languages possible through KPO or external tools)	Primarily SQL and Python (other languages supported by some target systems)	Typically a combination of Python for initial transformations in Airflow and SQL/Python in the target system
Option to use distributed computing	Only if external tools are used for transformations	Yes	Yes

Choose a method for passing data between tasks

In ETL and ELT pipelines, it is common that one task needs to access data created by a previous task. For example in an ETL pipeline the **extract** task might fetch records from an API and the **transform** task needs to run transformations on these records. At every point in your pipeline, whether it uses an ELT, ETL or combined pattern, whenever you pass data between your tasks, you need to decide whether to use XCom or explicitly write to external storage.

XCom stands for Cross-Communication and it is Airflow's way of passing information between tasks. Originally, XCom was developed to store small amounts of data, for example a filepath or a configuration in JSON format, in the Airflow metadata database. This data can be retrieved from any task in the same Airflow instance, even tasks in different DAGs.

The original implementation of XCom, referred to today as *standard XCom backend* (see Figure 4), can only handle very small amounts of data, and is restricted to types of data it can serialize to JSON or a limited number of other formats, for example Pandas dataframes (as of Airflow 2.6).

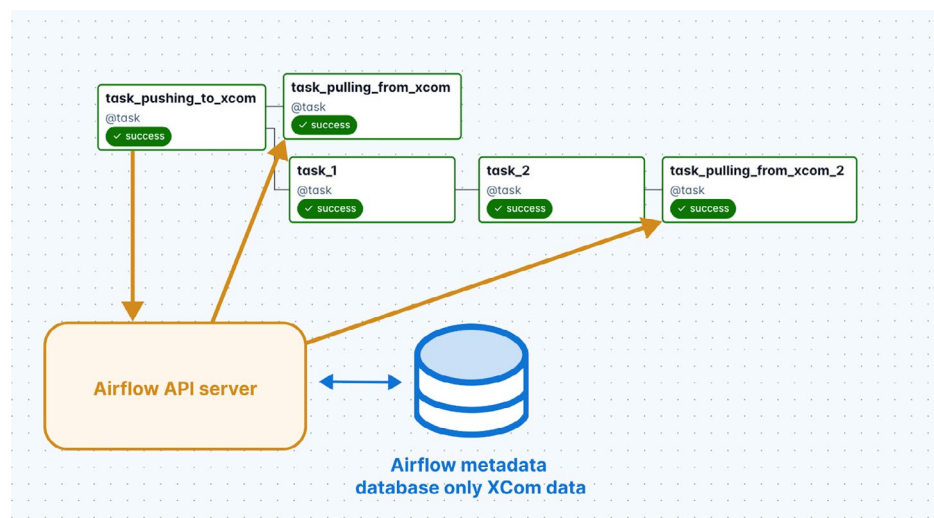


Figure 4. Standard XCom backend. Data is serialized and written to the Airflow metadata database.

As Airflow evolved, the need to pass large amounts of data from task to task arose and with it came *custom XCom backends*.

When using a custom XCom backend, the data is stored in a system external to Airflow, typically an object storage solution like Amazon S3, GCS, or Azure blob storage (see Figure 5). The Airflow metadata database only saves a string referencing the data location, typically a URI.

Custom XCom backends allow data to be saved between tasks in any location and format, removing limitations on data size and type.

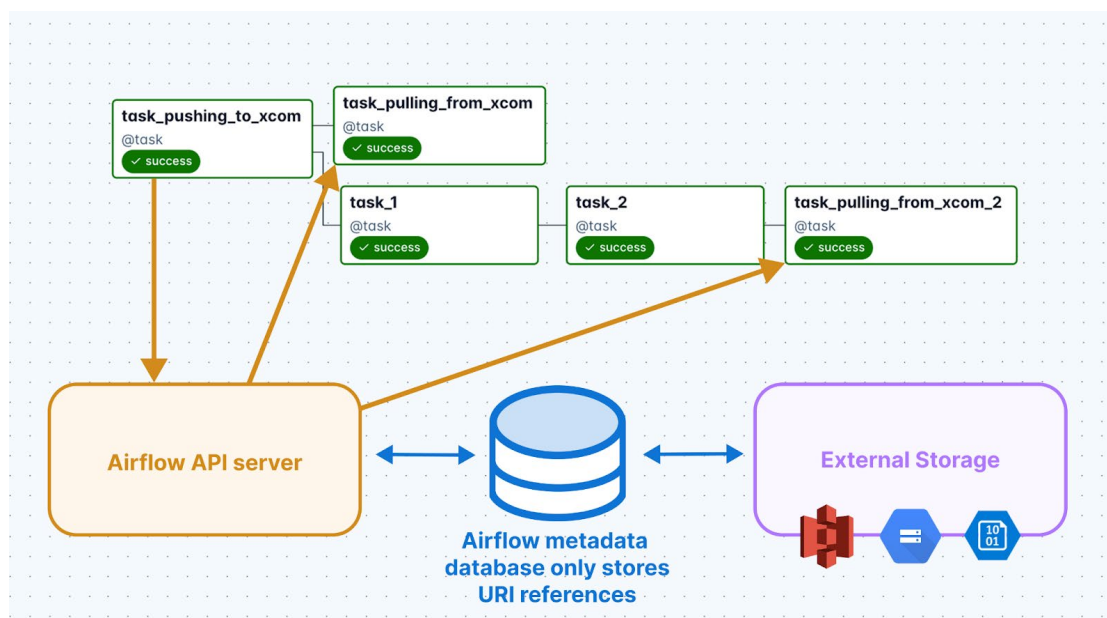


Figure 5. Custom XCom backend. Data is serialized and written to external storage, the Airflow metadata database only stores a reference to the data location.

There are two main ways you can create a custom XCom backend:

- **Create your own custom XCom backend class**, see [our documentation](#) for a code example.
- **Use the Object Storage XCom backend**, the easy way to set up a custom XCom backend using only environment variables. See [Object Storage XCom backend](#), for more information.

The other common pattern is to explicitly write your data to external storage from within an Airflow task using any tool to interact with the external storage solution. This pattern gives you complete flexibility over what data is saved where and in which format from each task.

The table below lists some aspects to consider when deciding between using a custom XCom backend and explicitly writing to external storage. This decision can be made independently for every time data is passed between tasks, which means that again, combined patterns are common.

helpful hint

It is a best practice to use a custom XCom backend in production environments if any significant amounts of data is passed through XCom, which has become simple to set up with the Object Storage XCom backend.

A frequently used pattern is to have a custom XCom backend handling most of the data passed between tasks and choose explicit writing and retrieving of data from external storage for tasks with custom requirements for serialization or data location.

	Custom XCom backend	Explicit external storage
Implementation options	Object Storage XCom backend Custom XCom backend class	Direct writing and retrieving of data from inside the Airflow task
Data location	Usually an object storage solution: S3, GCS, Azure blob storage	Usually an object storage solution: S3, GCS, Azure blob storage
Can store large amounts of data	Yes	Yes
Reference to the data location stored in the Airflow metadata database	Yes	No
Storing data from different tasks in different systems	Needs custom implementation in custom XCom backend class	Yes
Data can be accessed by multiple downstream tasks, including in other DAGs	Yes	Yes
Use with traditional operators	Yes	Only if implemented in an operator, you will likely need to write a custom operator

At this point, you might wonder: “If I need additional setup to pass data between tasks, why not just perform all operations—ET and L—within a single Python script in one task?”

While this approach is possible in Airflow, it’s generally considered best practice to break down your pipeline into separate, smaller tasks - a concept known as **atomic tasks**. By isolating each step of the process into its own task, you gain much greater visibility into what’s happening at each stage of the pipeline. This becomes especially valuable when something goes wrong, as you can pinpoint the exact step where the issue occurred.

Moreover, having an atomic structure enables you to **retry** or **rerun** only the failed task without having to repeat earlier steps, which can save time and avoid the overhead of reprocessing costly or time-consuming transformations.

Airflow's compute vs external compute

The **T** in ETL and ELT stands for any kind of **Transformation** of the data. These transformations can range from simple dropping of extra metadata that was retrieved together with the desired data from an API to complex aggregations of data from multiple sources requiring distributed compute.

Airflow allows full flexibility in choosing where to run transformations and most can be achieved using Python inside the Airflow worker with two limitations:

- If your data is larger than the storage of your Airflow worker, you should consider alternative transformation options. Astro customers can leverage larger workers using the [Worker Queues feature](#).
- If your transformations are complex to the point of necessitating distributed compute, you cannot do them in Airflow workers. You can either provision external resources like [Apache Spark](#) or [Ray](#), or transform your data in a target system supporting distributed computing.

Your choice of ETL vs ELT influences which compute options you can leverage, as you can see below.

	ETL	ELT
Transform data in the Airflow worker using Python	+	+
Transform data in the Airflow worker using another language via the KPO	+	+
Use native services to transform data in the source system	+	-
Provision external resources for distributed computing, e.g. Spark, Ray	+	+
Use SQL running in the target system's compute	-	+
Use a transformation tool like dbt Core leveraging target system compute	-	+

helpful hint

If you use the popular transformation tool dbt Core, you can leverage a special provider package, [Astronomer Cosmos](#), to create Airflow DAGs based on dbt Core projects. This open-source provider package was developed by Astronomer and has become one of the most popular ways to run dbt Core with Airflow. See [Orchestrating dbt with Apache Airflow® using Cosmos](#) for more information.

Task-oriented vs asset-oriented approach to writing pipelines

Apache Airflow 3 offers two different approaches to writing DAGs:

- 1. Task-oriented approach:** Define a DAG object (using the `@dag` decorator) and fill it with tasks.
- 2. Asset-oriented approach:** Use the shorthand `@asset` to define one DAG containing one task. When the task completes successfully it will also produce an event to an [Airflow Asset](#).

The table below shows a side-by-side comparison of the two approaches. Generally most pipelines can be written using either approach and which to pick comes down to personal preference. Note that assets are under development and additional features such as [asset partitioning](#) are planned for coming minor releases.

	Task-oriented approach	Asset-oriented approach
Mindset	“Action” first - you are thinking about what each step in your pipeline does.	“Data first” - you are thinking about the data each step in your pipeline produces.
Boilerplate Code	A little bit, but tasks can be reused across DAGs in a modular approach.	Minimal
Adapt pipelines to your data at runtime	Possible with dynamic task mapping	Not possible in Airflow 3.0.
Build data-driven pipelines	Within a DAG, tasks can have complex dependencies between them and DAGs can be connected using data-driven scheduling .	Assets can be scheduled depending on each other using the same data-driven scheduling practices.
Dependency visualization	Task dependencies are shown in graph view for each Airflow DAG, cross-DAG dependencies created through assets are shown in asset graphs.	Dependencies between assets are shown in asset graphs.

Task-oriented approach

The task-oriented approach is the most common way to define Airflow pipelines and the only possible way to write pipelines in versions before Airflow 3.

To write a task-oriented pipeline, you need to define one or more DAG objects using the `@dag` decorator and then fill those DAGs with tasks.

You have several options for how to define the tasks, see [Task-oriented approach: choice of operators and decorators](#). The code snippet below shows how to define a simple DAG with three tasks using the `@task` decorator.

```
from airflow.sdk import dag, task, chain
from pendulum import datetime

@dag(
    start_date=datetime(2025, 8, 13),
    schedule="@daily",
)
def my_dag():
    @task
    def extract():
        return {"a": 1, "b": 2}

    _extract = extract()

    @task
    def transform(data):
        return sum(data.values())

    _transform = transform(_extract)

    @task
    def load(data):
        print(data)

    _load = load(_transform)

    chain(_extract, _transform, _load)

my_dag()
```

You can see the dependencies between the 3 tasks in the graph view of the Airflow UI.

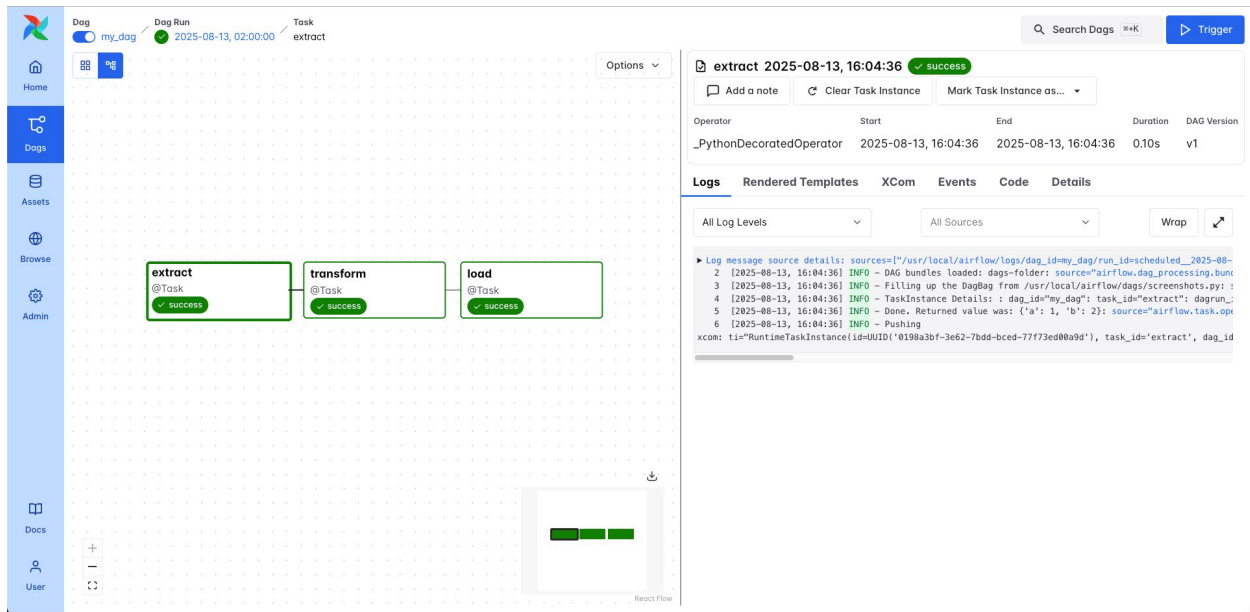


Figure 6. Graph view of the simple `my_dag` DAG consisting of 3 tasks.

If you have two DAGs created with the task-oriented approach you can create a data-driven dependency between them using the `Asset` object. In a nutshell you can use `Asset` like flags to give to individual tasks to raise as soon as they finish successfully and then schedule your DAGs to run on one or more flags being raised.

The code snippet below shows two DAGs that depend on each other with a data-driven schedule.

```
from airflow.sdk import dag, task, Asset
from pendulum import datetime
```

```
@dag(
    start_date=datetime(2025, 8, 13),
    schedule="@daily",
)
def my_etl_dag():
    @task(outlets=[Asset("ETL_done")])
    def producer_task():
        pass

    producer_task()
```

```
my_etl_dag()
```

```
@dag(
    schedule=[Asset("my_etl_dag")]
)
def my_ml_dag():
    @task
    def placeholder_task():
        pass

    placeholder_task()
```

```
my_ml_dag()
```

You can see dependencies between DAGs that are created via an Asset in asset graphs that are listed under the **Assets** tab in the Airflow UI.

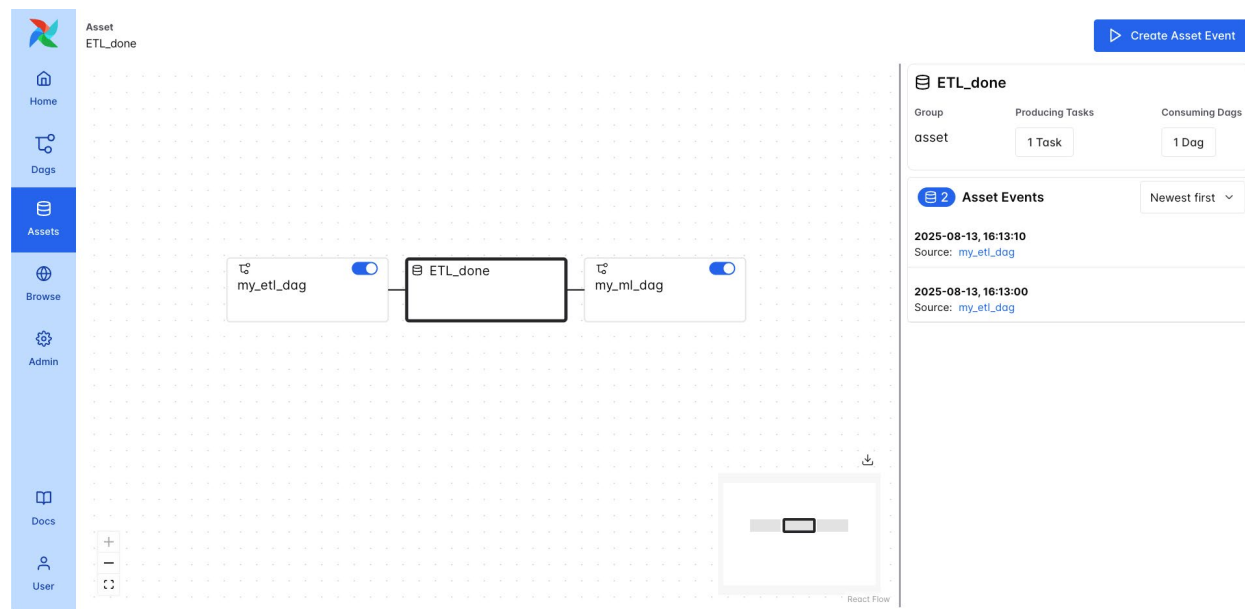


Figure 7. Asset graph of two DAGs depending on each other via the **ETL_done** asset.

You can learn more in the section on [data-driven scheduling](#).

Asset-oriented approach

The asset-oriented approach, which is new in Airflow 3 allows you to quickly define a DAG containing one task that updates one Asset with minimal boilerplate code.

The code above creates one DAG with the DAG ID **my_asset_1** containing one task with the task ID **my_asset_1** that produces updates to the Airflow Asset **my_asset_1**.

```
from airflow.sdk import asset

@asset(schedule=None, tags=["basic_asset_example"])
def my_asset_1():
    print("Add any Python Code!")
```

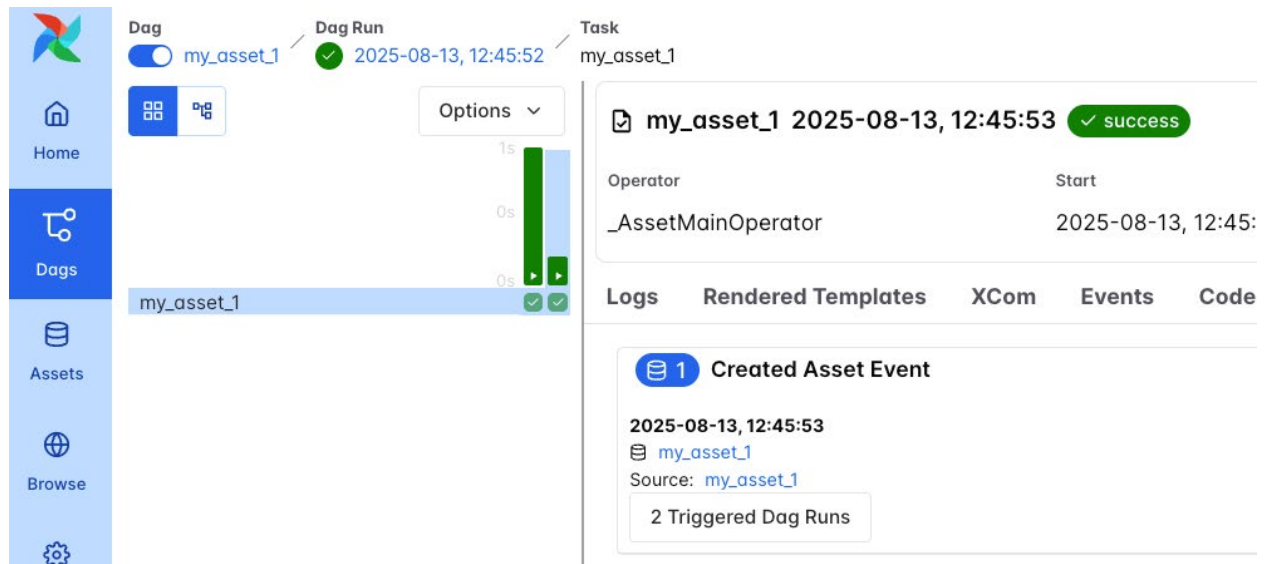


Figure 8. DAG created with an `@asset` decorator on a function named `my_asset_1`. Note how the ID of the DAG, of the singular task and the `Asset` object are all the same.

Assets can run on a schedule like regular DAGs, including data-aware schedules, i.e. an `@asset` can be scheduled based on updates to `Assets`.

It is also possible to push data to XCom from assets and retrieve them using a cross-DAG XCom pull, as well as attach small amounts of `Metadata` to the extra dictionary of the Asset event.

The following code snippet shows several assets that depend on each other, push to and pull from XCom and use the extra dictionary of the Asset event. You can find the example [here on GitHub](#).

```
from airflow.sdk import asset, Metadata, Asset

@asset(schedule=None, tags=["basic_asset_example"])
def my_asset_1():
    print("Add any Python Code!")

@asset(schedule=[my_asset_1], tags=["basic_asset_example"])
def my_asset_2():
    # you can return data from an @asset decorated function
    # to push it to XCom
    return {"a": 1, "b": 2}

@asset(schedule=[my_asset_1, my_asset_2], tags=["basic_asset_example"])
def my_asset_3(context):
    # you can pull the data from XCom via the context
    # note that this is a cross-dag xcom pull
    data = context["ti"].xcom_pull(
        dag_id="my_asset_2",
        task_ids=["my_asset_2"],
```

```

        key="return_value",
        include_prior_dates=True,
    )[0]

    print(data)

@asset(schedule=[my_asset_1], tags=["basic_asset_example"])
def my_asset_4(context):
    # you can also attach Metadata to the Asset Event
    yield Metadata(Asset("my_asset_4"), {"a": 1, "b": 2})

# note that using Asset("my_asset_4") is equivalent to passing the my_asset_4
# function directly. This is handy if your assets are defined in separate
# Python files.
@asset(schedule=[Asset("my_asset_4")], tags=["basic_asset_example"])
def my_asset_5(context):
    # and retrieve the Metadata from the Asset Event
    metadata = context["triggering_asset_events"][Asset("my_asset_4")][0].extra
    print(metadata)

```

In the Airflow 3 UI you can view asset dependencies by selecting a graph from the list shown under the Assets button. This is the same view that shows you asset based dependencies of DAGs written with the task-oriented approach. You can even create dependencies between DAGs written with the task-oriented and with the asset-oriented approach.

The following **asset** graph is created by the example code above.

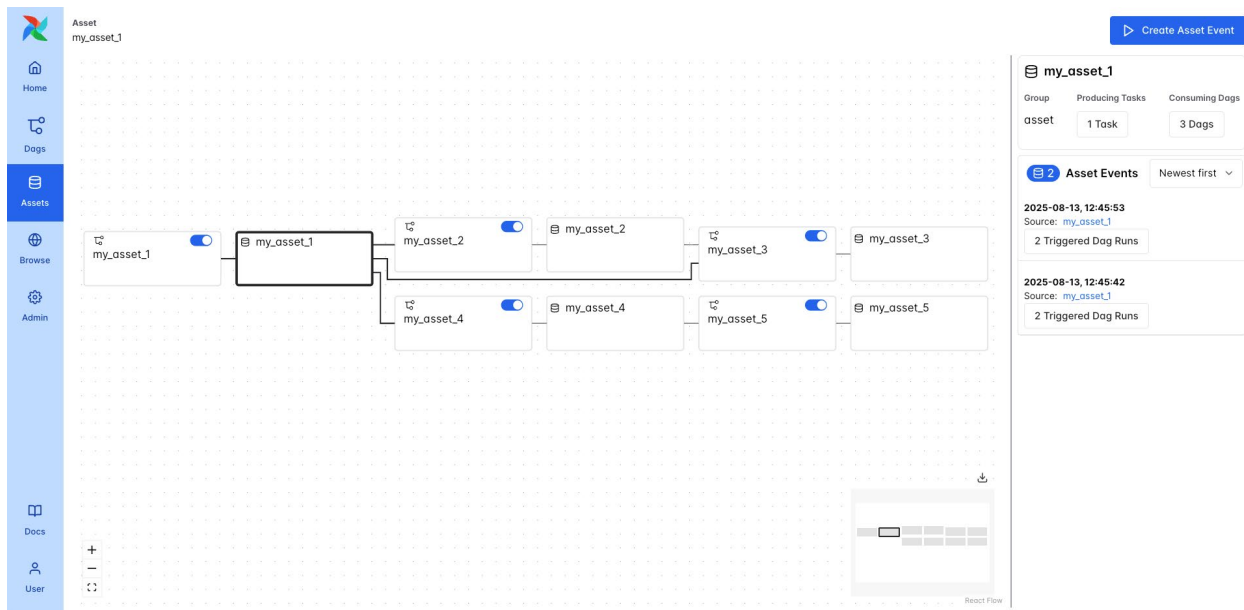


Figure 9. Asset graph created from the 5 **@assets** defined in the code snippet above.

Task-oriented approach: choice of operators and decorators

In Airflow you can turn any Python function into a task by using the `@task` decorator or the `PythonOperator`. Whether to use the decorator or operator is a question of personal choice, with the decorator offering some ease of use relating to [passing data between tasks](#).

```
# from airflow.sdk import task

@task
def my_task():
    # any code
    return "hello :)"

# from airflow.providers.standard.operators.python import PythonOperator

def _my_function():
    # any code
    return "hello :)"

my_task = PythonOperator(
    task_id="my_task",
    python_callable=_my_function
)
```

It is common to use [Airflow Hooks](#), which connect to external systems such as databases, inside your custom functions to retrieve credentials from [Airflow connections](#).

For some common ETL and ELT tasks, predefined operators exist. These operators are Python classes that interact with another tool, such as a source system, target system or transformation tool. A commonly used example is the `SQLExecuteQueryOperator` from the [Common SQL provider package](#) that can interact with many SQL-based target systems to run SQL statements in them.

```
# from airflow.providers.common.sql.operators.sql import
SQLExecuteQueryOperator

get_data = SQLExecuteQueryOperator(
    task_id="get_data",
    conn_id="my-database-connection",
    sql="SELECT * FROM MY_SCHEMA.MY_TABLE"
)
```

If predefined operators exist for your use case it is a best practice to use them, since they are maintained and tested by the community. You can use the [Astronomer registry](#) to find provider packages containing relevant operators for the system you are interacting with. Some commonly used provider packages are mentioned in the [Key Airflow providers for ETL and ELT](#) section of this eBook.

There are some operators, sometimes referred to as transfer operators, that perform both extraction and loading in the same task. For example, the [CopyFromExternalStageToSnowflakeOperator](#) extracts data from S3 and directly loads it to Snowflake using a Snowflake stage, bypassing loading of the data into the Airflow worker.

If no operator exists for your use case but you would still like to standardize how tasks in your ETL and ELT pipelines are handled in your organization, consider writing a custom operator. See [our documentation for template code](#).

For long running tasks, you can write custom deferrable operators, see [Create a deferrable operator](#).

You can mix operators and decorators in the same DAG and [freely pass data between them using XComs](#).

Data Quality Checks

Every pipeline inherently has one built-in data quality check: your stakeholder messaging you about errors—if you’re unlucky, late on a Friday evening.

To avoid relying on this last-resort “check,” it’s wise to design your pipelines with programmatic data quality checks in mind.

There are two fundamentally different types of data quality checks:

- **Stopping Checks:** These checks will stop the pipeline from progressing if they fail. Stopping checks are typically applied when a data issue could negatively impact downstream tables, such as duplicates in a primary key column.
- **Warning Checks:** These checks send an alert or warning to a stakeholder but allow the pipeline to continue. They are often used for domain-specific concerns, such as when a field exceeds a certain threshold of null values, indicating potential data issues.

Data quality checks can be slotted between any of your tasks in an ETL or ELT pipeline, with the most common positions being right after data extraction, to run basic checks on the source data, and after loading to leverage SQL-based data quality checks.

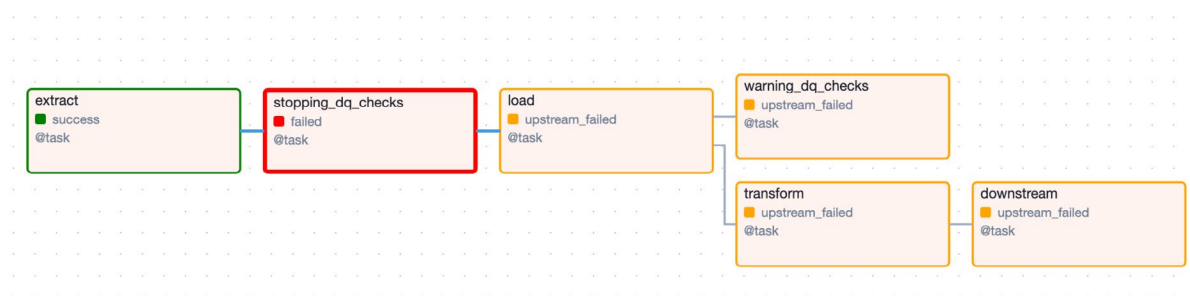


Figure 6. A stopping data quality check failing and causing the pipeline to halt.

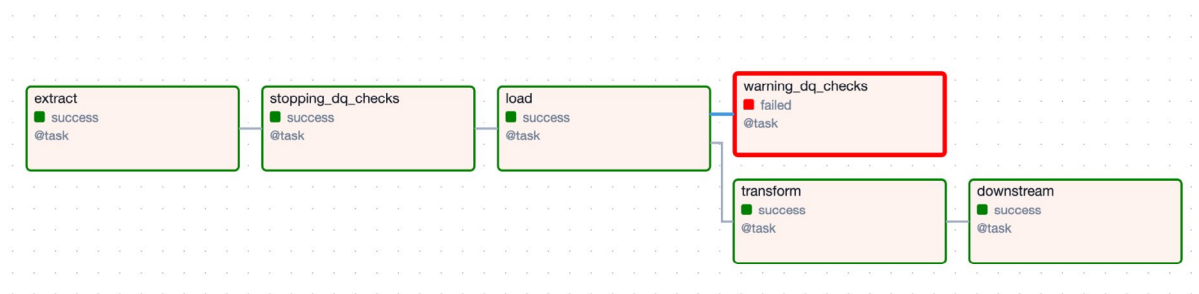


Figure 7. A warning data quality check failing while the pipeline continues.

Many data engineers also opt for a pattern where data is loaded into a temporary table first and only added to the permanent table when it passes its data quality checks, see this [advanced ETL DAG with data quality checks](#) for an example.

When designing data quality checks it is a good practice to unleash your inner pessimist: what's the worst way your data could go wrong, and what are the most severe downstream consequences?

Consider how difficult it would be to recover from faulty data making its way into an application—especially if it's user-facing, like prices on an e-commerce site. Additionally, assess the potential cost of downstream processes that run on bad data. Sometimes, a stopping data quality check can prevent costly reprocessing that would otherwise be required due to a quality issue.

Another important aspect to keep in mind is avoiding alert fatigue. While it's great to have comprehensive data quality checks, too many alerts and warnings may lead stakeholders to ignore your emails. It's crucial to discuss with your stakeholders (and yourself) which alerts truly matter and require action. No diagnostics without implications for treatment!

Data quality checks can be defined a number of ways, some of the most popular ones are:

- Using the SQL check operators on data in a relational database. See the [Run data quality checks using SQL check operators](#) tutorial.
- Defining your own custom checks using Python code.
- Using a data quality tool like [GreatExpectations](#) or [Soda](#).
- Defining your own custom checks using SQL.
- Run data quality checks with dbt test.

helpful hint

Another option is to define data quality checks running on your data independently of your DAGs in [Astro Observe](#), Astronomers' observability platform.

Airflow best practices for ETL and ELT

Regardless of your answers to the decision questions in the previous sections, there are a few best practices to always keep in mind when it comes to writing ETL and ELT pipelines.

Three core DAG writing best practices

There are three best practices to follow, no matter the purpose of your Airflow pipeline:

- **Atomicity:** Atomicity refers to designing each task to complete one specific function in the pipeline. In an ETL or ELT pipeline, each task in your DAG, ideally accomplishes one step: extracting, transforming, loading or an ETL adjacent task such as sending a report. This atomic structure gives you full observability over your pipeline and enables you to rerun only failed tasks without having to repeat earlier steps, saving time and compute.
- **Idempotency:** This is just a fancy way of saying “Same in - same out”. When creating an Airflow pipeline, you ideally have idempotent tasks and DAGs, meaning with the same input your task always has the same output. In ETL pipelines it is common to use timestamps to partition your data, when using Airflow you can pull different timestamps relating to the DAG run time from the [Airflow context](#) like for example the `logical_date` to partition your data, ensuring that the partition stays the same even if the DAG run is rerun months later.
- **Modularity:** Don't repeat yourself - this software engineering best practice holds true for Airflow DAGs. Since DAGs are defined in Python code, you can modularize your functions to be imported into several DAGs. And if you'd like to standardize how your team interacts with tools you can write your own [custom operator classes](#).

A best practice closely related to modularity is to [treat your DAGs like config files](#). Try to avoid having supporting code like long SQL statements or Python scripts inside your DAG code. Instead, modularize them in a separate location like an include folder to make the DAG easier to read and enable reuse of your scripts across DAGs.

ETL DAG design best practices

When creating DAGs for ETL and ELT there are some additional best practices you should consider.

Avoid top-level code

By default Airflow parses all DAG files every 30 seconds. During this parse all top-level code, meaning all code outside of Airflow tasks, is executed. If you have long-running code at the top-level, it can cause Airflow to slow down, and if you make connections and calls to external systems you can even start incurring costs!

This is the pattern you should **avoid**:

```
"""WARNING: BAD PRACTICE"""

...

# Bad practice: top-level code in a DAG file
hook = PostgresHook("postgres_conn")
results = hook.get_records("SELECT * FROM my_table;")

@dag(
    ...
)
def bad_dag():
    ...
```

A common reason users resort to top-level code when connecting to databases is the need to dynamically generate tasks. For instance, they may want to create one loading task per table in a schema but do not know the number or names of the tables in advance. A best practice for creating pipelines that can adapt to such dynamic data at runtime is to use [Dynamic Task Mapping](#), which avoids top-level code and provides full visibility into the task copies created during each DAG run.

Automatic retries

Airflow allows you to define how often a task should be retried automatically before it fails, as well as how long to wait between retries by using the **retries** and **retry_delay** task parameters. Retries are especially important to make ETL and ELT pipelines that might run into concurrency limitations of databases more resilient.

You can set retries for your whole Airflow environment by using a config variable:

```
AIRFLOW__CORE__DEFAULT_TASK_RETRIES=5
```

If you want all tasks in one of your DAGs to retry a different number of times you can override the config at the DAG level using the **default_args** dictionary and at the task level by using task parameters.

```
# from airflow.sdk import dag
# from datetime import timedelta

@dag(
    ...
    default_args={
        "retries": 3,
        "retry_delay": timedelta(seconds=30)
    },
)
```

```
# from airflow.sdk import task

# from datetime import timedelta

@task(retries=2, retry_delay=timedelta(minutes=10))

def extract():

    # your code

    return "hi"
```

For more information on Airflow retries, see [our documentation](#).

Use Software Development Lifecycle best practices

Airflow offers the great advantage of being able to define your pipelines as Python code and you should treat your DAG code, including supporting code like SQL statements, like any application code.

This means you should:

- Store your code in a **version control system** like [git](#).
- Use at least two **environments**, a development and production environment.
- Use **tests** on your DAG code.
- Deploy your code with automated testing in a **CICD** pipeline.

If you are not familiar with version control systems, see [git's entry level explanation videos](#). In short, version control allows you to track changes to your code, making it possible to reverse specific edits or merge updates from different developers working on separate branches.

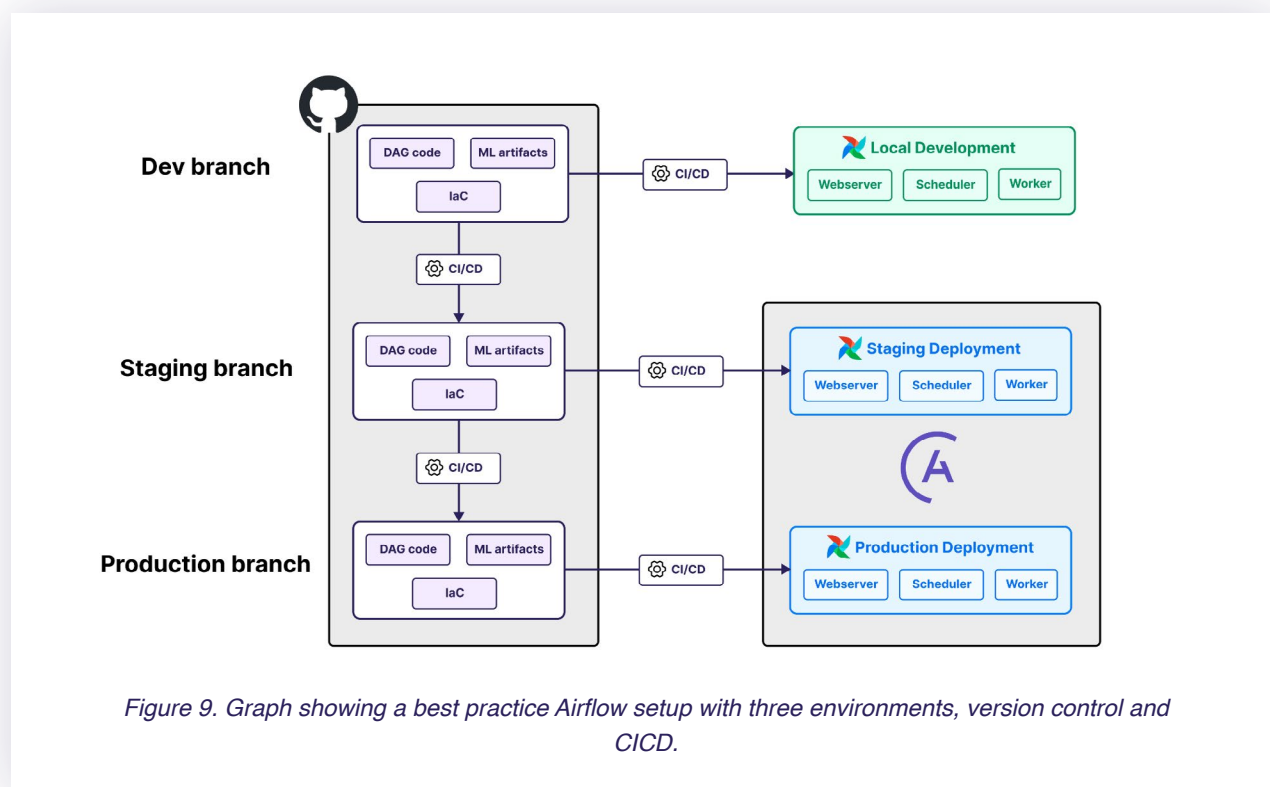
When creating ETL and ELT pipelines with Airflow you should have at least two branches for your code, the development (dev) branch where you make changes to test in a local development environment and the production (prod) branch where your code gets promoted to after thorough testing.

Deploying from the production branch to the production environment should be automatic. Astro customers can use the [GitHub integration](#) or our [CICD template scripts](#).

As soon as your DAGs become more complex, we recommend having at least three environments:

- **Dev**: mapped to your local development environment.
- **Staging**: mapped to a cloud environment that serves as end-to-end testing.
- **Production**: mapped to a cloud environment that powers your data products in production.

When your code changes are promoted from a lower to a higher environment they go through a CI/CD process including automated testing.



Testing Airflow code

In Airflow you have different layers of code for which you can write tests:

- **Unit Tests**: All custom Python code should be unit tested as you would in any software application code.
- **DAG validation tests**: You can define Airflow-specific tests to ensure your DAGs meet organizational standards, such as requiring tags on DAGs or restricting the use of certain operators.
- **Integration Tests**: Since Airflow interacts with many tools it is wise to have end-to-end integration tests, these typically run as pipelines in a staging environment.

Note that all these tests are about the DAG *code* itself, the data flowing through your pipeline is tested by specialized tasks in your pipeline using [Data Quality Checks](#).

If you use the [Astro CLI](#) to run your DAGs, you can add all of your tests to the **tests** folder and run them with the command **astro dev pytest** (no matter which Python framework you are using for your tests).

Unit Tests

In Airflow you can create unit tests the same way as you would for any Python code with any Python testing framework.

You should test all your custom operators, as well as all Python functions used in your tasks. If you use operators from provider packages you don't need to write unit tests for them, since these operators are already tested by the community.

This is an example of a unit test for a custom operator doing basic math created with the [unittest](#) framework. The test makes sure that if given the inputs **2** and **3** as well as the operation **+**, the operator will return **5**. You can find the full test suite for this [basic custom operator](#) in the corresponding testing file [here](#).

```
import unittest

from include.custom_operators import MyBasicMathOperator


class TestMyBasicMathOperator(unittest.TestCase):

    def test_addition(self):

        operator = MyBasicMathOperator(

            task_id="basic_math_op", first_number=2, second_number=3, operation="+"

        )

        result = operator.execute(None)

        self.assertEqual(result, 5)
```


DAG validation tests

DAG validation tests examine your DAGs according to your defined standards. You import the DAGs using the Airflow **DagBag** class and create one test for each property you want to enforce, using any Python testing framework.

The following is an example of using **pytest** to only allow only one decorator (**@task**) and one operator (**SQLExecuteQueryOperator**).

```
import logging

import os

from contextlib import contextmanager

import pytest

from airflow.models import DagBag


@contextmanager
def suppress_logging(namespace):
    logger = logging.getLogger(namespace)

    old_value = logger.disabled

    logger.disabled = True

    try:
        yield
    finally:
        logger.disabled = old_value


def get_dags():
    """
    Generate a tuple of dag_id, <DAG objects> in the DagBag
    """
```

```

with suppress_logging("airflow"):
    dag_bag = DagBag(include_examples=False)

def strip_path_prefix(path):
    return os.path.relpath(path, os.environ.get("AIRFLOW_HOME"))

return [(k, v, strip_path_prefix(v.fileloc)) for k, v in dag_bag.dags.items()]

ALLOWED_OPERATORS = [
    "_PythonDecoratedOperator", # this allows the @task decorator
    "SQLExecuteQueryOperator",
]

@pytest.mark.parametrize(
    "dag_id, dag, fileloc", get_dags(), ids=[x[0] for x in get_dags()]
)

def test_dag_uses_allowed_operators_only(dag_id, dag, fileloc):
    """
    Test if a DAG uses only allowed operators.
    """
    for task in dag.tasks:
        assert any(
            task.task_type == allowed_op for allowed_op in ALLOWED_OPERATORS
        ), f"{task.task_id} in {dag_id} ({fileloc}) uses {task.task_type}, which is not in the list of allowed operators."

```

Scaling

As soon as you start moving from local development to production, you need to make sure your environment scales with your pipelines.

On the DAG authoring side this means you need to be aware of scaling configurations and parameters. There are ways to adjust how Airflow scales at three different levels:

- **Environment-level settings:** These configuration parameters apply to your whole Airflow instance. A common value you might need to adjust here is the `AIRFLOW__CORE__DAGBAG_IMPORT_TIMEOUT`. By default DAG processing will time out after 30 seconds, causing tasks to not execute in larger environments. Increasing the value of this environment variable can prevent the DAG import process from timing out. You can find all available Airflow configurations in the [Configuration Reference](#).
- **DAG-level settings:** These DAG parameters allow you to affect how much a specific DAG can scale, for example if you have a long running DAG that should only ever have one concurrent active DAG run, you can set `max_active_runs=1`. See [DAG parameters](#) for a comprehensive list of dag-level parameters.
- **Task-level settings:** For individual tasks it is possible to control their execution at the task-level using task parameters. A common pattern is to assign a set of tasks to an [Airflow pool](#), which limits how many tasks in this set can run concurrently.

You can learn more about available scaling parameters in [this guide](#).

Airflow environment best practices

The other part of Airflow that needs to scale to keep up with your use case is the underlying infrastructure. Deploying Airflow on your own infrastructure is possible, but can be tricky and necessitates advanced Kubernetes or Docker knowledge. The other option is to use a managed service like Astronomer that handles the infrastructure for you. You can try it out with a [14-day free trial](#).

No matter how you run Airflow, it is a good best practice to upgrade to new Airflow versions as soon as possible as each of them contains bug and security related fixes as well as new features. Note also that you should upgrade provider packages separately by pinning their versions, that way you can track all version changes and potential effects in your Airflow environment.

Upgrading Airflow environments should be done programmatically and go through the same CI/CD processes as code changes. Astro customers also have the option to test out an upgrade and [roll back to a previous deployment](#) version in case there are any issues.

Key Airflow features for ETL and ELT

Apache Airflow offers a lot of features that allow you to write advanced ETL and ELT pipelines. In this section, you'll learn about some of the key features that are essential for building more complex ETL and ELT DAGs, supported by code snippets to help you put them into action.

These are some of the most important Airflow features for ETL and ELT:

- **Dynamic Task Mapping:** A way to create pipelines that adapt to your data at runtime.
- **Defining DAGs dynamically:** Dynamic DAGs refers to the possibility in Airflow to create DAG code programmatically.
- **Modularized task groups:** Modularize commonly used task patterns for efficient and standardized DAG development.
- **Data-driven scheduling:** With Airflow assets you can run DAGs based on a combination of successful completion of tasks anywhere in your Airflow environment.
- **Airflow Object Storage:** A newer feature to simplify interacting with Object Storage solutions such as Amazon S3, Google Cloud Storage or Azure Blob Storage. This feature also includes the **Object Storage XCom backend**.
- **Other Airflow features for ETL and ELT:** This section covers other relevant Airflow features at a high-level linking out to in-depth guides for each of them.

This section assumes you have a foundational understanding of Apache Airflow. If you come across any terms or concepts that are unfamiliar with, don't worry—refer to the [Airflow Crash Course](#) for a quick refresher before diving back in!

Dynamic Task Mapping

Static Airflow pipelines can handle many ETL and ELT use cases, but often, you'll reach a point where you need a more fine-grained, parallelized structure in your DAG.

A common scenario is loading data from multiple locations in object storage to a target database. In a static pipeline, you might have one task that extracts all the data, one that transforms it, and one that loads it. However, if just one of the source files is corrupted, the entire task fails and must be rerun.

A better approach is to create a separate task for each storage location. This way, if a failure occurs, you only need to rerun the pipeline for the specific location that contained the corrupted file.

But how do you design such a pipeline when the number of locations or their names change frequently?

This is where dynamic task mapping can help you.

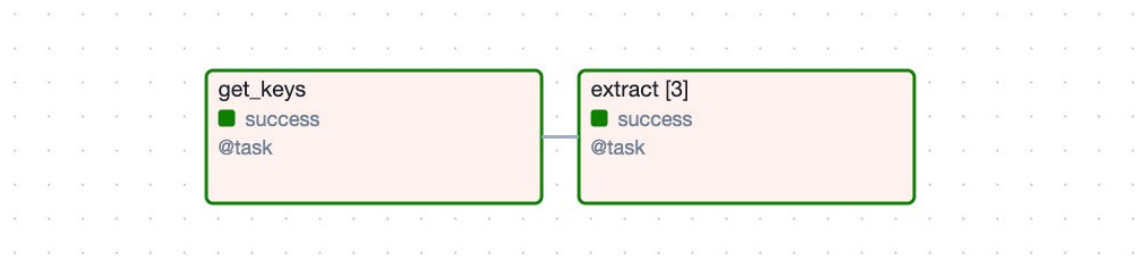


Figure 10. A DAG graph containing a dynamically mapped task called **extract** with three parallel dynamically mapped task instances (**[3]**).

Dynamic task mapping allows you to define a task as a placeholder to create a changing number of dynamically mapped task instances for every DAG run, based on input received at runtime.

In our example you have one upstream task (**get_keys**) pulling the list of this DAG run's extraction locations and then passing that list to the dynamically mapped task to create one dynamically mapped task instance per element in the list, in Figure 10, 3 copies of the **extract** task.

In its most basic form, dynamic task mapping uses two methods:

- **.expand()**: This method passes the parameter that you want to map. A separate parallel task is created for each input. For some instances of mapping over multiple parameters, **.expand_kwargs()** is used instead.
- **.partial()**: This method passes any parameters that remain constant across all mapped tasks which are generated by **expand()**.

The code snippet below shows how the **extract** task can be given a value for one of its parameters **object_storage_type** that stays constant across all mapped task instances, and a changing value for its other parameter **key** that is derived from the upstream task **get_keys**.

This simple example hardcodes the return values from **get_keys** but in a real life scenario it could change with every DAG run, for example at one run at 1am returning 20 keys, at the next run at 2am returning 1000 keys and at its next run at 3am returning no keys at all.

No matter how many elements the **get_keys** task returns, the **extract** task can adapt.

```

@task

def get_keys():

    # Code to fetch object storage locations

    return ["bucket1", "bucket2", "bucket3"]


@task

def extract(object_storage_type, key):

    # Code to extract data from object storage locations

    return f"{object_storage_type}://{key}"


_get_keys = get_keys()

_extract = extract.partial(object_storage_type=s3).expand(key=_get_keys)

```

There are many variations to dynamic task mapping, including mapping over multiple outputs or whole task groups. For more information see [our documentation](#).

See [ELT DAG using Dynamic Task Mapping](#) for a full DAG code example.

Generate DAG code programmatically

Since DAGs are just Python code they can be created programmatically.

In an ETL or ELT pipeline you often have a similar DAG structure interacting with different source and target systems. A common practice is to have one DAG per combination of source and target system. If you find yourself writing the same code over and over, it might be time to create your DAG code programmatically instead.

A popular way to generate DAG code is using [dag-factory](#). This package allows you to simplify code and reduce redundancy by allowing DAG definitions in YAML format, as shown in the example below.

```
my_dag:

  schedule: '0 0 * * *'

  default_args:

    owner: 'Astro'

    start_date: '2025-08-01'

  tasks:

    extract:

      operator: airflow.operators.python.PythonOperator

      python_callable_name: extract

      python_callable_file: /usr/local/airflow/include/python_func.py

    transform:

      operator: airflow.operators.python.PythonOperator

      python_callable_name: transform

      python_callable_file: /usr/local/airflow/include/python_func.py

      dependencies: [extract]

    load:

      operator: airflow.operators.python.PythonOperator

      python_callable_name: load

      python_callable_file: /usr/local/airflow/include/python_func.py

      dependencies: [transform]
```

When dag-factory is installed in an Airflow environment, you can generate DAGs from yaml files like this with the following DAG generation methods:

```
import os

from pathlib import Path

from dagfactory import load_yaml_dags

DEFAULT_CONFIG_ROOT_DIR = "/usr/local/airflow/dags/"
CONFIG_ROOT_DIR = Path(os.getenv("CONFIG_ROOT_DIR", DEFAULT_CONFIG_ROOT_DIR))

config_file = str(CONFIG_ROOT_DIR / "my_dag_config_file.yaml")

load_yaml_dags(
    globals_dict=globals(),
    config_filepath=config_file,
)
```

See [this tutorial](#) for in-depth instructions on how to use the dag-factory.

There are other methods to programmatically create DAG code that allow for more customization than the dag-factory, see [this guide](#) for more information.

Modularized task groups

Some tasks are happier in groups! Well, maybe, but in any case, task groups help you to navigate DAGs in the Airflow UI and can be used to modularize common task patterns in your DAGs.

If you have a common pattern of tasks, consider creating a Python class representing that pattern inside a task group that can be instantiated with just a few parameters.

For example, imagine you have the same extract, transform, load pattern as a part of many of your DAGs where only a few parameters change, like for example a URL and a key of interest. You can standardize the pattern in a custom class like the **MyETLTaskGroup** shown in the code below.

```
from airflow.sdk import TaskGroup, task

class MyETLTaskGroup(TaskGroup):
    """A task group containing a simple ETL pattern."""

    def __init__(self, group_id, url: str, key_of_interest: str, **kwargs):
        super().__init__(group_id=group_id, **kwargs)

        self.url = url

    @task(task_group=self)
    def extract(url: str) → dict:
        """Extracts data from a URL.

        Args:
            url (str): The URL to extract data from."""

        import requests

        response = requests.get(url)

        return response.json()
```

```

@task(task_group=self)

def transform(api_response: dict, key: str) → str:

    """Transforms the api_response.

    Args:

        api_response (dict): The number to transform."""

    return api_response[key]

@task(task_group=self)

def load(transformed_data: str):

    """Prints the transformed data (simulating loading).

    Args:

        transformed_data (dict): The transformed data."""

    print(transformed_data)

load(transform(api_response=extract(self.url), key=key_of_interest))

```

This class can be imported and used in several DAGs to create a three task pattern in just one line of code.

```

from airflow.sdk import dag, task

from include.custom_task_group.etl_task_group import MyETLTaskGroup

@dag

def simple_etl_task_group():

    @task

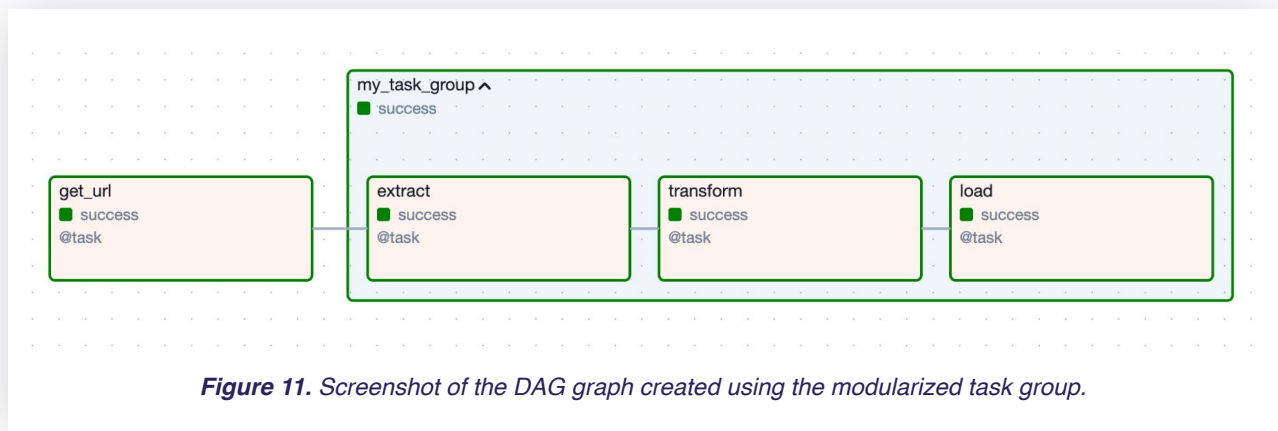
    def get_url():

        return "https://catfact.ninja/fact"

```

```
MyETLTaskGroup(group_id="my_task_group", url=get_url(), key_of_interest="fact")
```

```
simple_etl_task_group()
```



You can learn more about task groups, including custom modularized task groups in [this guide](#) and [this webinar](#).

Data-driven scheduling

Airflow offers extensive [scheduling capabilities](#), with one of the most versatile being data-driven scheduling using Airflow assets. Data-driven scheduling allows a DAG to run automatically, whenever a supporting asset is updated by any task within your Airflow environment.

In ETL and ELT pipelines it is common to have several downstream DAGs performing actions on the data loaded into a database, these DAGs can be scheduled to run exactly once all their required assets have been updated, with an asset schedule.

To use data-driven scheduling you first need to define an **Asset** and give it a name. The name of the asset is its unique identifier.

```
# from airflow.sdk import Asset

my_asset = Asset("my_asset_one")
```

In any task updating this asset, for example by uploading a new file to a bucket, you can set the **outlets** parameter to tell Airflow about the task-asset relationship. Note that this functions as a flag defined by you, the DAG author, Airflow does not interact with the underlying asset.

```
@task(outlets=[Asset(my_asset)])

def load():

    # Code to extract data from object storage locations

    pass

load()
```

```
# from airflow.providers.google.cloud.transfers.gcs_to_gcs import GCSToGCSOperator

load = GCSToGCSOperator(
    task_id="load",
    ...
    outlets=[Asset(my_asset)]
)
```

To schedule a downstream DAG based on an update to one or more assets, set its schedule parameter to the asset identified by the same name.

```
@dag(
    start_date=datetime(2025, 8, 13),
    schedule=[Asset(my_asset)],
)
```

You can also use conditional logic to combine assets in a DAG schedule, for example to run a DAG if either of two assets have been updated.

```
@dag(
    start_date=datetime(2024, 10, 18),
    schedule=(my_asset | my_other_asset),
    # Use () instead of [] to be able to use conditional asset scheduling!
)
```

You can even combine asset scheduling with time-based scheduling, creating a DAG that will run both, at a set time and additionally whenever its asset condition is fulfilled. The DAG below runs every week, 2am UTC on Monday, as well as whenever one of two assets has received an update.

```
# from airflow.timetables.assets import AssetOrTimeSchedule

# from airflow.timetables.trigger import CronTriggerTimetable

@dag(
    start_date=datetime(2024, 10, 18),

    schedule=AssetOrTimeSchedule(
        timetable=CronTriggerTimetable("0 0 * * 1", timezone="UTC"),
        assets=(my_asset | my_other_asset),
    ),
)
```

Dependencies between DAGs and assets can be explored in the Airflow UI in asset graphs found under Assets in the Airflow UI.

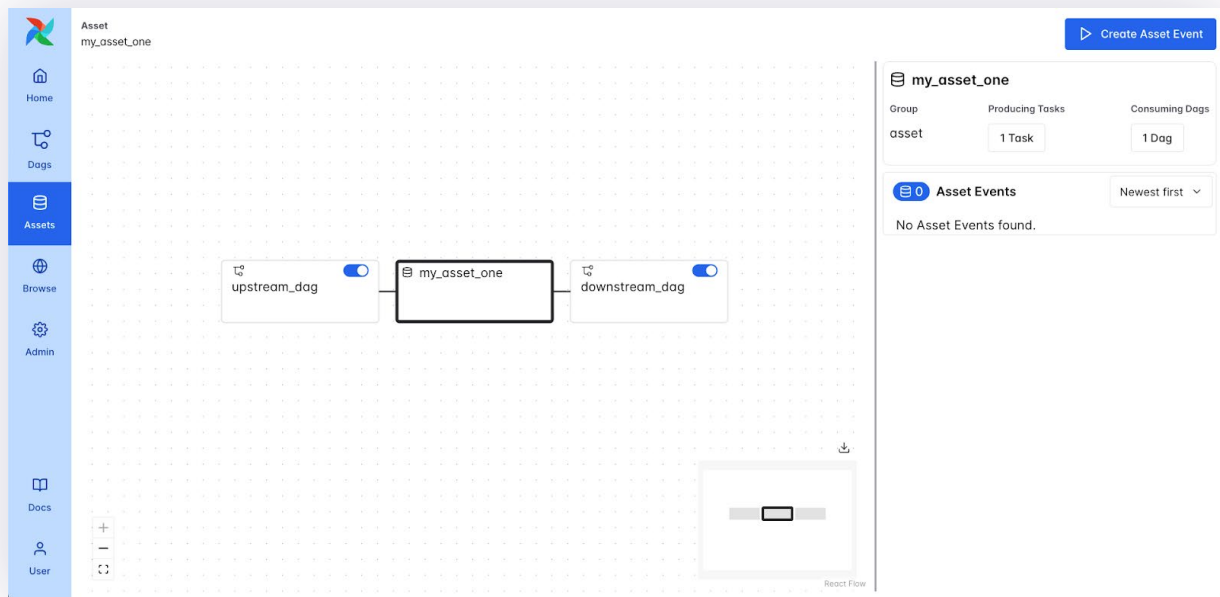


Figure 14. Screenshot of an asset graph showing one asset with an upstream and downstream DAG dependency.

To learn more about data-driven scheduling including advanced features, see [this guide](#).

Airflow Object Storage

The [Airflow Object Storage](#) provides a generalized API to interact with different object storage solutions. Since many ETL and ELT pipelines extract data from object storage services like S3, GCS or Azure Blob storage, using Airflow Object Storage can simplify your pipelines significantly.

The key aspects of this feature are:

- You can abstract your interactions with object stores using a [Path API](#). Note that some limitations apply due to the nature of different remote object storage systems. See [Cloud Object Stores are not real file systems](#) for more information.
- You can switch between different object storage systems without having to change your DAG code.
- You can transfer files between different object storage systems without needing to use XToYTransferOperator operators.
- It transfers large files efficiently. For object storage, Airflow uses [shutil.copyfileobj\(\)](#) to stream files in chunks instead of loading them into memory in their entirety.

Additionally, the Airflow Object Storage feature allows you to create a [custom XCom backend](#) only by setting a few environment variables. You can find an example in the code snippet below that will create a custom XCom backend in the [my-bucket](#) bucket in S3 where any XCom larger than 1000 bytes will be stored, compressed in zip format. Note that you need to have the [Common IO provider](#) installed to use this feature.

```
AIRFLOW_CONN_AWS_DEFAULT='{
    "conn_type": "aws",
    "login": "<YOUR AWS KEY ID>",
    "password": "<YOUR AWS SECRET KEY>"
}'

AIRFLOW__CORE__XCOM_BACKEND="airflow.providers.common.io.xcom.backend.
XComObjectStorageBackend"

AIRFLOW__COMMON_IO__XCOM_OBJECTSTORAGE_PATH="s3://my_aws_conn@my-bucket"

AIRFLOW__COMMON_IO__XCOM_OBJECTSTORAGE_THRESHOLD="1000"

AIRFLOW__COMMON_IO__XCOM_OBJECTSTORAGE_COMPRESSION="zip"
```

You can learn more about the Airflow Object Storage feature in [this tutorial](#) and about the Object Storage custom XCom backend in [this tutorial](#).

Other important features for ETL and ELT

In addition to the features covered above there are several other Airflow features commonly used in ETL and ELT pipelines:

- **Airflow Trigger Rules:** Sometimes you want tasks to run, even if the upstream task was unsuccessful or got skipped, Airflow trigger rules allow you detailed control over what upstream conditions allow a task to run.
- **Branching in Airflow:** Branching in Airflow offers the possibility to decide which task(s) to run next based on conditions at runtime.
- **Sensors:** Sensors are special types of operators that wait for a specific condition to be met before proceeding, for example for a file to land in an object storage bucket. You can use pre-existing sensors or create your own.
- **Deferrable operators:** Deferrable operators use asynchronous processes running in the Airflow Triggerer component, this frees up a worker slot while waiting for long running actions in external systems to complete. Many deferrable operators perform similar actions to sensors. You can use pre-existing deferrable operators or **create your own**.

Key Airflow providers for ETL and ELT

While in Airflow you can perform any action in any other tool that has an API using `@task` decorated tasks with direct API calls to your source system, sometimes it is nice to have some of that work done for you. This is where Airflow providers come in; they help you to connect to many popular data tools.

You can find an overview of different providers and their modules in the [Astronomer Registry](#). This section calls out some of the most commonly used ones in ETL and ELT pipelines.

Major Cloud Providers

ETL and ELT pipelines often interact with data and services in different cloud provider systems. You can find relevant operators, hooks and more in the related cloud provider packages:

- **Amazon Provider**: Offers modules for AWS services like S3 and Redshift.
- **Google Provider**: Provides modules to orchestrate Google services, including GCS and BigQuery.
- **Microsoft Azure Provider**: Includes modules for Azure Blob Storage and other Azure services.

To interact specifically with cloud-based object storage systems, you can use the **Common IO Provider**. It contains modules from the **Airflow Object Storage** feature, offering a unified API to interact with files across different clouds.

Transformation tools

Although Airflow workers can handle many transformations, some transformations benefit from the computational power of external services like databases or distributed computing clusters.

Airflow contains provider packages for powerful computing options such as:

- **Apache Spark and Databricks**: Good for large-scale data processing and advanced analytics. See the [ELT with Apache Airflow® and Databricks reference architecture](#) for example code.
- **Ray and Anyscale**: Ideal for distributed data transformations and machine learning workloads.
- **Kubernetes (via `KubernetesPodOperator`)**: Allows task execution within a Kubernetes cluster for resource-intensive tasks.
- **Cosmos by Astronomer**: Cosmos enables orchestration and observability of dbt projects in the Airflow UI, displaying individual models as separate tasks in DAGs or task groups for improved monitoring.

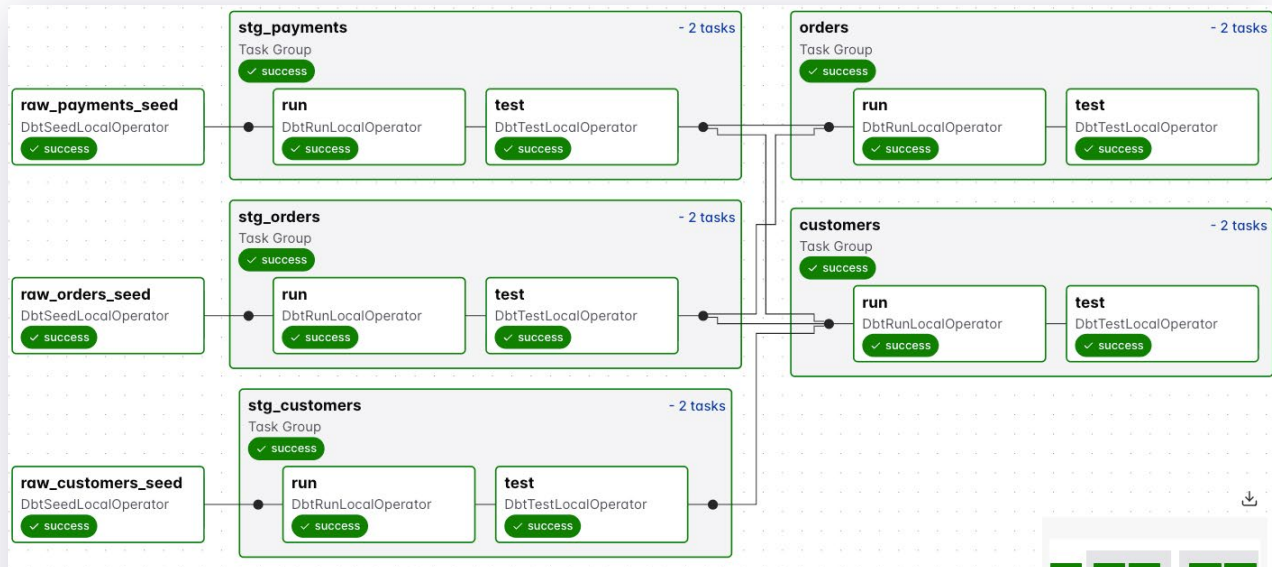


Figure 15. A dbt project rendered as an Airflow dag using Cosmos.

Data warehouses and data lakes

Almost every ETL and ELT pipeline interacts with a data warehouse or data lake. Aside from the cloud-native solutions like Redshift on AWS, BigQuery on GCS or Azure SQL on Azure, for which you can find modules in the relevant cloud provider packages, there are a couple of other notable providers that make it easier for you to interact with databases.

- The **common SQL provider** includes operators that can interact with many different SQL databases, based on where the provided `conn_id` points. This provider includes the `SQLExecuteQueryOperator` that has replaced deprecated database-specific operators such as the `PostgresOperator` or the `SnowflakeOperator`.
- The **Snowflake provider** offers operators to interact with Snowflake. See the [ELT with Snowflake and Apache Airflow® for eCommerce](#) reference architecture for example code.
- The **Postgres provider** contains the `PostgresHook` used in the DAGs of the [companion repository](#).

Airflow for ETL/ELT Checklist

You've learned a lot in this book! This checklist contains some of the key Airflow best practices for ETL and ELT and can serve as a cheat sheet for implementing your next pipeline.

Before you start coding:

- ☐ Complete the [pre-flight checklist](#)
- ☐ Check the [Astronomer registry](#) for relevant providers for your use case
- ☐ Set up a [version control and CI/CD](#) with a separate development and production environment
 - ☐ **Local dev:** we recommend using the free and open-source [Astro CLI](#)
 - ☐ **Production:** we recommend Astro with a free trial

DAG writing best practices for every DAG:

- ☐ [Avoid top-level DAG code](#)
- ☐ Make sure your tasks are: [atomic, idempotent and modular](#)
- ☐ Set [automatic retries](#) for your tasks
- ☐ [Test your DAG code](#) like any other software code
- ☐ [Scale your Airflow environment](#) to keep up with your needs

Key Airflow features to use in ETL and ELT pipelines:

- ☐ Use a [custom XCom backend](#) if you want to pass larger amounts of data between your tasks.
- ☐ Use [data quality checks](#) in your pipelines to prevent faulty data from reaching downstream systems
- ☐ Use [Dynamic Task Mapping](#) to create tasks that adapt to your data at runtime ("One task per X").
- ☐ Schedule your DAGs based on updates to the [assets](#) they depend on
- ☐ Use [deferrable operators](#) for long running tasks
- ☐ Create [modularized task groups](#) for common patterns in your DAGs

Why Choose Astronomer for ETL and ELT

	Astronomer	Traditional ETL/ELT Tools
All ETL/ELT stages orchestrated on one platform	✓	-
Fully customizable Python-based workflows	✓	-
Kubernetes-based scalability	✓	-
Extensive library of pre-built modules to interact with popular tools in the data stack	✓	✓
Built on a successful, community driven open-source project	✓	-
Advanced dbt integration	✓	-
Comprehensive pipeline-level data observability with SLA dashboards , real-time alerts , and full lineage tracking	✓	-
Efficient resource management ensures high performance and quick adaptation to workload changes	✓	-
Implement comprehensive data validation and quality checks within your workflows	✓	✓
Lower costs by consolidating multiple tools and leveraging an open-source foundation	✓	-
Built-in security features like encryption, RBAC, and SSO	✓	✓

Conclusion

We hope this ebook has sparked new ideas for building and enhancing your ETL and ELT pipelines.

You've explored:

- The **definitions of ETL and ELT** and how these patterns are implemented in practice.
- A **pre-flight checklist** to ensure you go down the right path once you start coding.
- **Key DAG writing best practices** for ETL and ELT pipelines, including the application of software development practices and testing strategies.
- **Cutting-edge Airflow features** that can elevate your ETL and ELT pipelines, such as dynamic task mapping, data-driven scheduling, and custom XCom backends with the Airflow Object Storage feature.

What's next?

Beyond a lot of happy coding (hopefully!), you could consider deploying your DAGs to a [free Astro trial](#).

If you're looking to dive deeper into Airflow, check out one of our recorded [webinars](#) where we explore the newest Airflow topics or sign up to the [Astronomer Academy](#) to go through a structured Airflow course and get Airflow certified!

Thank you for reading, and have a lovely day!

ELT & ELT ON ASTRO

Simplify Your Workflows with Astronomer

When you use Astro to manage your ETL and ELT pipelines, you can unlock the full potential of your data and drive insightful business decisions with ease.

Get Started Free

Talk to an Expert

ASTRONOMER